

Отчёт по практической работе №1

Группа ЕТ-442 Замятин Андрей Александрович

1 Задание

Требовалось написать на языке Python 3 программу, которая создает и обучает нейронную сеть для распознавания рукописных цифр с использованием метода градиентного спуска и базы данных MNIST. Работа выполнена по указаниям к практической работе 1.

2 Исходные данные

Сеть содержит три слоя. Входной слой принимает изображения размером 28 на 28 пикселей. Это дает 784 входа. Скрытый слой содержит 30 нейронов. Выходной слой содержит 10 нейронов по числу цифр от 0 до 9. Использованы язык Python версии 3.14.4 и библиотека NumPy ветки 2.4. Версия Python видна в строке состояния редактора на скриншоте в разделе 6. Признак версии NumPy виден на том же скриншоте в виде предупреждения про NumPy 2.4. Файл данных mnist.pkl.gz заранее загружен и находится в папке practical-1 рядом с программой, так как загрузчик открывает его по относительному имени.

3 Состав программы

Программа состоит из двух модулей. Модуль network.py описывает класс Network и функции активации. Модуль mnist_loader.py загружает файл mnist.pkl.gz и приводит наборы к виду, удобному для обучения. Набор обучения содержит 50000 пар входного изображения и ожидаемого вектора. Вектор ожидаемого ответа имеет длину 10 и содержит единицу в позиции верной цифры. Набор проверки содержит 10000 пар. Набор тестирования содержит 10000 пар. Такое разделение соответствует структуре базы MNIST.

4 Алгоритм

Для активации использована сигмоидальная функция. Она вычисляется через экспоненту из NumPy в векторизованной форме и применяется поэлементно. Ее производная выражена через значение самой функции по формуле $s'(z) = s(z)(1 - s(z))$, где $s(z)$ это сигмоида. В коде функция sigmoid_prime вызывает sigmoid(z) дважды, поэтому экспонента фактически вычисляется дважды. Код не сохраняет промежуточное значение сигмоиды для повторного использования.

Прямое распространение последовательно вычисляет потенциалы слоев как произведение матрицы весов на сигналы предыдущего слоя с добавлением смещения. Затем к потенциалам применяется активация. Одного вызова метода прямого распространения достаточно для получения ответа сети.

Обучение выполнено методом стохастического градиентного спуска. В начале каждой эпохи обучающая выборка перемешивается случайным образом. Затем она делится на малые подвыборки размером 10. Для каждой подвыборки вызывается один шаг обновления весов и смещений.

Градиенты для каждой пары из подвыборки вычисляются методом обратного распространения. Сначала выполняется прямой проход с запоминанием потенциалов и активаций всех слоев. Ошибка выходного слоя находится как разность выхода сети и ожидаемого вектора, умноженная на производную активации. Ошибка скрытых слоев находится через транспонированные матрицы весов со сдвигом по слоям назад. Затем для каждого слоя вычисляются градиенты по смещениям и по весам. Градиенты всех пар подвыборки суммируются.

Весы и смещения обновляются вычитанием среднего градиента, умноженного на скорость обучения 3.0. После завершения эпохи вызывается оценка точности на тестовом наборе. Ответом сети считается номер выходного нейрона с наибольшей активацией. Подсчитывается число верных ответов из 10000.

5 Запуск в используемой системе

В указаниях описан запуск через среду IDLE в Windows с ручной установкой рабочего каталога. Фактическая работа выполнена в редакторе VS Code запуском из терминала интерпретатором Python напрямую без IDLE.

Текущий каталог предварительно установлен в каталог с файлами программы и базой данных. Это важно, так как загрузчик открывает файл `mnist.pkl.gz` по относительному имени. Проверка показала 50000 пар в обучении и по 10000 пар в проверке и тестировании. Инициализация сети с топологией 784, 30, 10 прошла без ошибок.

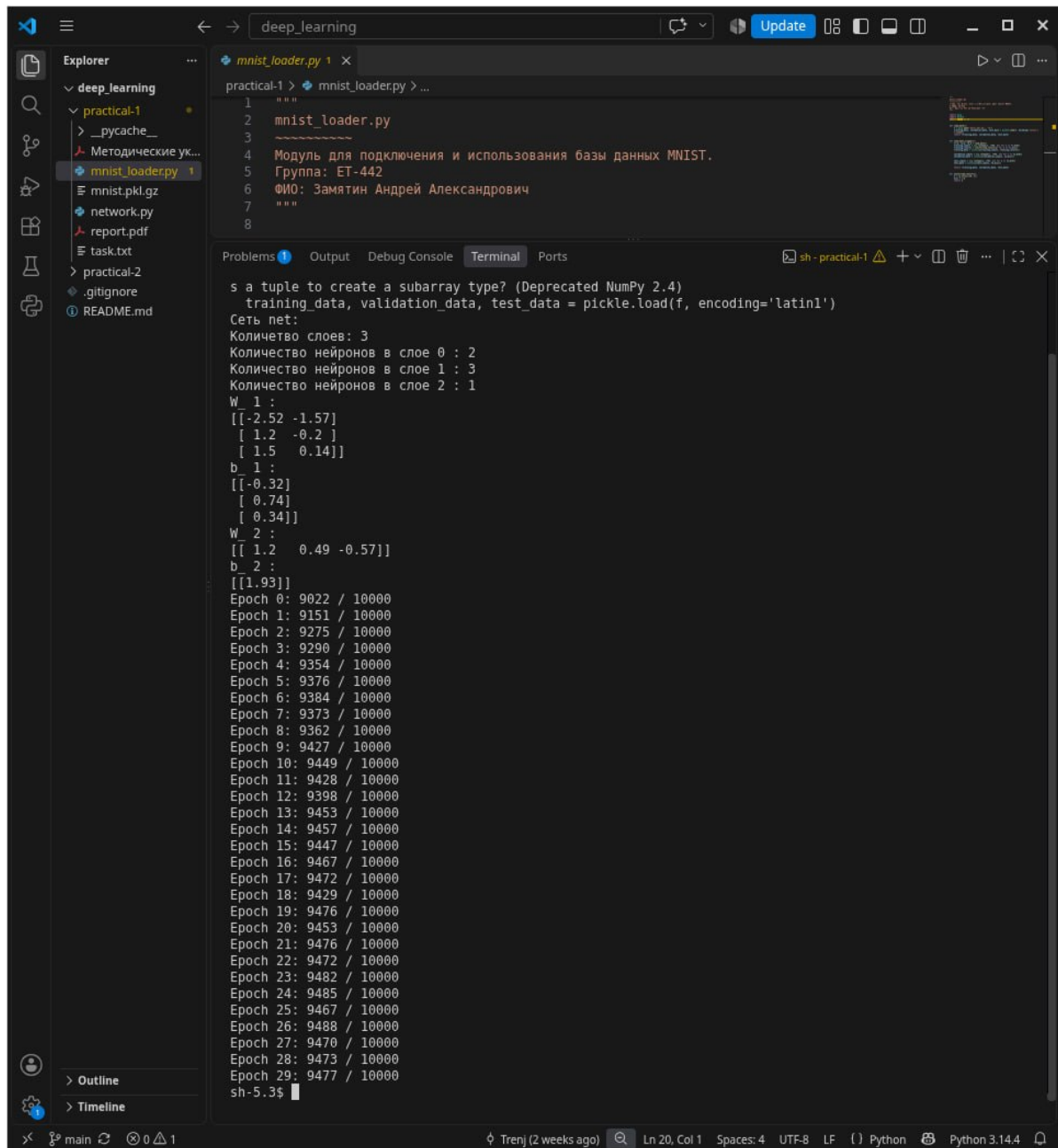
Для запуска обучения использована команда из файла `task.txt`:

```
python3 -c "import mnist_loader; training_data, validation_data, test_data =
mnist_loader.load_data_wrapper(); import network; net = network.Network([784, 30, 10]);
net.SGD(training_data, 30, 10, 3.0, test_data=test_data)"
```

При подключении модуля `network` выполняется тело программы из методички и на экран выводится информация о тестовой сети 2, 3, 1. Затем создается сеть 784, 30, 10 и запускается обучение. Обучение занимает несколько минут. В ходе обучения на экран выводится строка для каждой эпохи с номером эпохи и числом верных ответов из 10000.

6 Результат

Обучение выполнено за 30 эпох. Вывод программы полностью сохранен на скриншоте ниже. На нулевой эпохе получено 9022 верных ответа из 10000. Далее точность в целом росла с колебаниями. На последней эпохе с номером 29 получено 9477 верных из 10000. Наилучший результат 9488 верных из 10000 зафиксирован на эпохе с номером 26. Такой ход подтверждает корректность загрузки данных и работы методов обучения и оценки.



The screenshot shows a VS Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'deep_learning' with a subdirectory 'practical-1' containing files like 'mnist_loader.py', 'network.py', 'report.pdf', 'task.txt', and 'README.md'. The terminal shows the output of the 'mnist_loader.py' script, which includes a header, a tuple creation warning, network parameters, and a list of 30 epochs with their corresponding accuracy values.

```
1 """
2 mnist_loader.py
3 -----
4 Модуль для подключения и использования базы данных MNIST.
5 Группа: ET-442
6 ФИО: Замятин Андрей Александрович
7 """
8
9
10 s a tuple to create a subarray type? (Deprecated NumPy 2.4)
11 training_data, validation_data, test_data = pickle.load(f, encoding='latin1')
12
13 Сеть net:
14 Количество слоев: 3
15 Количество нейронов в слое 0 : 2
16 Количество нейронов в слое 1 : 3
17 Количество нейронов в слое 2 : 1
18 W_1 :
19 [[-2.52 -1.57]
20 [ 1.2 -0.2 ]
21 [ 1.5 0.14]]
22 b_1 :
23 [[-0.32]
24 [ 0.74]
25 [ 0.34]]
26 W_2 :
27 [[ 1.2 0.49 -0.57]]
28 b_2 :
29 [[1.93]]
30 Epoch 0: 9022 / 10000
31 Epoch 1: 9151 / 10000
32 Epoch 2: 9275 / 10000
33 Epoch 3: 9290 / 10000
34 Epoch 4: 9354 / 10000
35 Epoch 5: 9376 / 10000
36 Epoch 6: 9384 / 10000
37 Epoch 7: 9373 / 10000
38 Epoch 8: 9362 / 10000
39 Epoch 9: 9427 / 10000
40 Epoch 10: 9449 / 10000
41 Epoch 11: 9428 / 10000
42 Epoch 12: 9398 / 10000
43 Epoch 13: 9453 / 10000
44 Epoch 14: 9457 / 10000
45 Epoch 15: 9447 / 10000
46 Epoch 16: 9467 / 10000
47 Epoch 17: 9472 / 10000
48 Epoch 18: 9429 / 10000
49 Epoch 19: 9476 / 10000
50 Epoch 20: 9453 / 10000
51 Epoch 21: 9476 / 10000
52 Epoch 22: 9472 / 10000
53 Epoch 23: 9482 / 10000
54 Epoch 24: 9485 / 10000
55 Epoch 25: 9467 / 10000
56 Epoch 26: 9488 / 10000
57 Epoch 27: 9470 / 10000
58 Epoch 28: 9473 / 10000
59 Epoch 29: 9477 / 10000
60 sh-5.3$
```

Скриншот терминала с выводом обучения сети за 30 эпох

7 Вывод

Собрана и обучена трехслойная сеть с топологией 784, 30, 10 для входа 28 на 28. Все методы класса Network реализованы по указаниям, включая прямое распространение, стохастический градиентный спуск, обновление мини пакета, обратное распространение, оценку точности и производную стоимости. Загрузка данных MNIST выполнена корректно с приведением размерностей к виду 784 на 1 для входов и 10 на 1 для ответов. Обучение сходится, точность на тестовом наборе растет по эпохам, что видно по скриншоту. Отличие от текста указаний состоит в том, что функция load_data_wrapper возвращает list(zip(...)) вместо zip. Это безвредно, так как метод SGD все равно преобразует выборки в списки, и обучение проходит без ошибок. Подключение модуля network дополнительно печатает параметры тестовой сети 2, 3, 1, как предусмотрено телом программы в

указаниях.

Исходный код

Актуальная версия программы для данной практики хранится по адресу https://github.com/Trenj/deep_learning/tree/main/practical-1